ОТЧЕТ
по практической работе № 9
«Классы и объекты»

СОДЕРЖАНИЕ

1 ПОСТАНОВКА ЗАДАЧИ

1.1 Общая постановка задачи

Часть 1.

1) Реализовать (в отдельном файле) определение нового класса/типа данных: закрытые атрибуты, свойства, конструкторы, инициализация и вывод атрибутов.
2) Для демонстрации работы с объектами написать главную функцию, в которой создаются объекты класса и выводится информация, которая содержится в атрибутах.
3) Написать функцию, реализующую указанное в варианте действие. Рассмотреть два варианта: статическую функцию и метод класса. В основной функции продемонстрировать работу функции.
4) Используя статическую компоненту класса, подсчитать количество созданных в программе объектов.

Часть 2.

1) Добавить к реализованному классу указанные в варианте перегруженные операции.
2) Написать демонстрационную программу, в которой создаются объекты пользовательских классов и выполняются указанные операции.

Часть 3.

1) Реализовать класс-коллекцию (в отдельном файле), полем которого является одномерный массив (без использования стандартных коллекций C#) из элементов заданного в варианте типа. В классе реализовать: конструктор без параметров; конструктор с параметрами, заполняющий элементы случайными значениями; конструктор с параметрами, позволяющий заполнить массив элементами, заданными

пользователем с клавиатуры; метод для просмотра элементов массива.

2) Реализовать индексатор для доступа к элементам коллекции. Предусмотреть проверку выхода индекса за пределы массива.
3) Написать демонстрационную программу, позволяющую создать массив разными способами и распечатать элементы массива. Подсчитать количество созданных объектов.
4) Написать функцию поиска минимального значения в классе `Program` для выполнения указанного в варианте задания (использовать индексатор и, если необходимо, перегрузить нужные для выполнения задачи операции).

1.2 Постановка задачи для конкретного варианта

Класс: `Time`.

Поля: `int hours` (часы), `int minutes` (минуты).

Часть 1. Метод — вычитание минут (`int`) из объекта типа `Time`. Необходимо учесть, что количество минут в объекте не может быть больше 59 и меньше 0. Результат должен быть типа `Time`. Функция реализуется в двух вариантах: как статическая функция и как метод класса.

Часть 2. Перегружаемые операции:

- унарные операции:
  - `-` — обнуление часов и минут в объекте типа `Time`;
  - `--` — вычитание минут из объекта типа `Time` (учесть, что минут и часов не может быть меньше 0);
- операции приведения типа:
  - `int` (неявная) — результатом является количество часов (минуты отбрасываются);
  - `bool` (явная) — результатом является `true`, если часы и минуты не равны нулю, и `false` в противном случае;
- бинарные операции:
  - `==(Time t1, Time t2)` — результатом является `true`, если значения часов и минут левого операнда равны соответствующим значениям правого операнда, и `false` — в противном случае;

– `!=(Time t1, Time t2)` — результатом является `true`, если значения часов и минут левого операнда не равны соответствующим значениям правого операнда, и `false` — в противном случае.

Часть 3. Реализовать класс-коллекцию `TimeArray`, полем которого является одномерный массив `Time[] arr` элементов типа `Time`. В классе `Program` написать функцию поиска минимального элемента массива (с использованием индексатора и перегруженной операции сравнения <).

2 ДИАГРАММЫ КЛАССОВ

На рисунках **??**–**??** приведены диаграммы классов для каждой части работы. Символ «+» обозначает открытый член класса, «−» — закрытый; статические члены подчёркнуты.

2.1 Часть 1

Time
− hours : int = 0 − minutes : int = 0 − count : int − TotalMinutes : int {get} + Hours : int {get; set} + Minutes : int {get; set} + Count : int {get}
+ Time() + Time(h : int, m : int) + Time(other : Time) + ToString() : string + Print(prefix : string) : void + Subtract(time : Time, minutes : int) : Time + SubtractMinutes(minutes : int) : Time

Рисунок 1 – Диаграмма классов. Часть 1

2.2 Часть 2

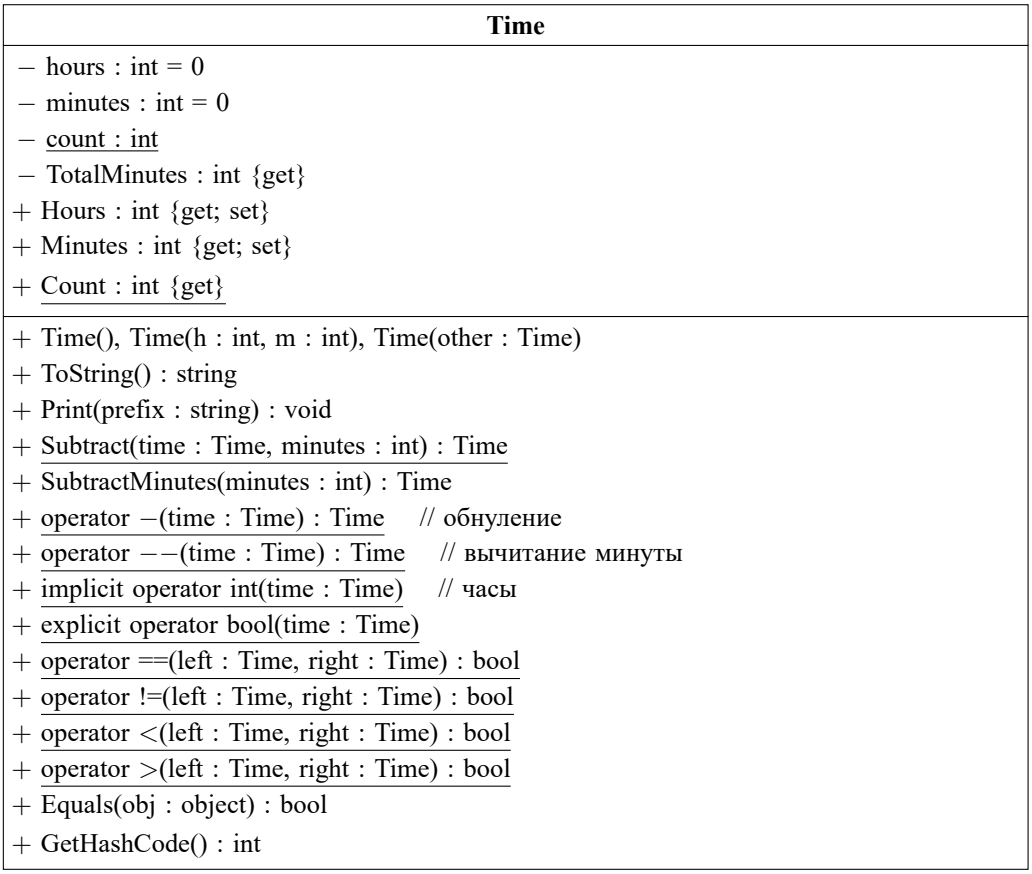


Рисунок 2 – Диаграмма классов. Часть 2

2.3 Часть 3

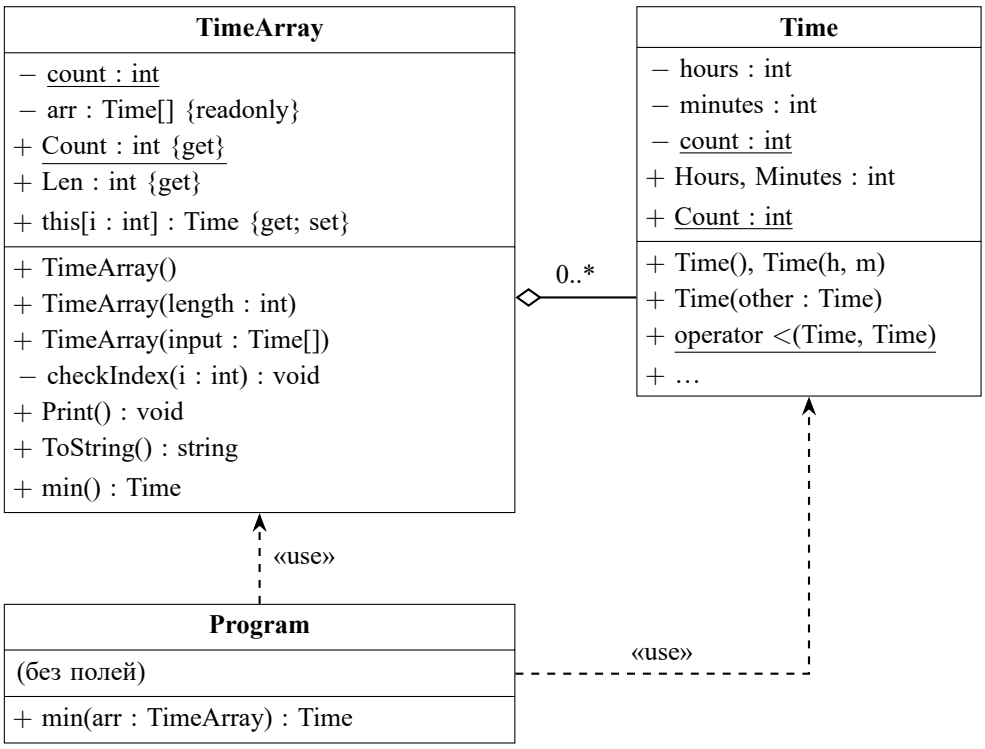


Рисунок 3 – Диаграмма классов. Часть 3

Класс `TimeArray` агрегирует массив объектов `Time`; класс `Program` использует оба класса при поиске минимального элемента. Вспомогательный статический класс `UserInterface` (ввод данных с клавиатуры и меню) на диаграммах не показан.

3 КОД ПРОГРАММЫ

Программа состоит из четырёх файлов: `Time.cs` (классы части 1 и 2), `TimeArray.cs` (класс-коллекция части 3), `UserInterface.cs` (ввод данных и меню) и `Program.cs` (главная функция, демонстрация, поиск минимума).

3.1 Файл Time.cs

```
using System;

namespace lab_10
{
    internal class Time
    {
        private int hours = 0;
        private int minutes = 0;

        private static int count;

        public static int Count
        {
            get { return count; }
        }

        public int Hours
        {
            get { return hours; }
            set
            {
                if (value < 0)
                {
                    throw new ArgumentOutOfRangeException("Ошибка! Часы
   не могут быть отрицательными");
                }
                hours = value;
            }
        }

        public int Minutes
        {
            get { return minutes; }
            set
            {
                if (value < 0 || value > 59)
                {
                    throw new ArgumentOutOfRangeException( "Ошибка!
   Минуты должны быть в диапазоне от 0 до 59");
```

```
                }
                minutes = value;
            }
        }

        private int TotalMinutes
        {
            get { return hours * 60 + minutes; }
        }

        public Time() {
            count++;
        }

        public Time(int h, int m)
        {
            Hours = h;
            Minutes = m;
            count++;
        }

        public Time(Time other)
        {
            Hours = other.Hours;
            Minutes = other.Minutes;
            count++;
        }

        public override string ToString()
        {
            return $"{Hours:00}:{Minutes:00}";
        }

        public void Print(string prefix)
        {
            Console.WriteLine($"{prefix} {this}");
        }

        public static Time Subtract(Time time, int minutes)
        {

            if (time.TotalMinutes < minutes)
            {
                throw new InvalidOperationException(
                    $"Из объекта {time} нельзя вычесть {minutes} мин:
результат был бы отрицательным");
            }
            int rest = time.TotalMinutes - minutes;
            return new Time(rest / 60, rest % 60);
        }
```

```
87
88          public Time SubtractMinutes(int minutes)
89          {
90              return Subtract(this, minutes);
91          }
92
93          public static Time operator -(Time time)
94          {
95              return new Time(0, 0);
96          }
97
98          public static Time operator --(Time time)
99          {
100             return Subtract(time, 1);
101         }
102
103         public static implicit operator int(Time time)
104         {
105             return time.Hours;
106         }
107
108         public static explicit operator bool(Time time)
109         {
110             return time.Hours != 0 || time.Minutes != 0;
111         }
112
113         public static bool operator ==(Time? left, Time? right)
114         {
115             if (left is null || right is null)
116             {
117                 return false;
118             }
119             return left.Hours == right.Hours && left.Minutes == right.
    Minutes;
120         }
121
122         public static bool operator !=(Time? left, Time? right)
123         {
124             if (left is null || right is null)
125             {
126                 return true;
127             }
128             return left.Hours != right.Hours && left.Minutes != right.
    Minutes;
129         }
130
131         public static bool operator <(Time left, Time right)
132         {
133             return left.TotalMinutes < right.TotalMinutes;
134         }
```

```
135
136         public static bool operator >(Time left, Time right)
137         {
138             return left.TotalMinutes > right.TotalMinutes;
139         }
140
141         public override bool Equals(object? obj)
142         {
143             return obj is Time;
144
145         }
146
147         public override int GetHashCode()
148         {
149             return HashCode.Combine(Hours, Minutes);
150         }
151
152     }
153 }
```

3.2 Файл TimeArray.cs

```
1  using System;
2  using System.Text;
3
4  namespace lab_10
5  {
6
7      internal class TimeArray
8      {
9          private static int count = 0;
10         private readonly Time[] arr;
11
12         public static int Count
13         {
14             get { return count; }
15         }
16
17         public int Len
18         {
19             get { return arr.Length; }
20         }
21
22         public Time this[int i]
23         {
24             get
25             {
26                 checkIndex(i);
27                 return arr[i];
```

```
28              }
29              set
30              {
31                  checkIndex(i);
32                  arr[i] = value;
33              }
34          }
35
36          private void checkIndex(int i)
37          {
38              if (i < 0 || i >= arr.Length)
39              {
40                  throw new ArgumentOutOfRangeException($"Индекс должен
    быть в диапазоне от 0 до {arr.Length - 1}");
41              }
42          }
43
44          public TimeArray()
45          {
46              count++;
47              arr = new Time[8];
48              for (int i = 0; i < Count; i++)
49              {
50                  this[i] = new Time();
51              }
52          }
53
54          public TimeArray(int length)
55          {
56              if (length <= 0)
57              {
58                  throw new ArgumentException("Длина массива должна быть
    больше нуля");
59              }
60              count++;
61              arr = new Time[length];
62              for (int i = 0; i < arr.Length; i++)
63              {
64                  this[i] = new Time(Random.Shared.Next(0, 24), Random.
    Shared.Next(0, 60));
65              }
66          }
67
68          public TimeArray(Time[] input)
69          {
70              count++;
71              arr = new Time[input.Length];
72              for (int i = 0; i < input.Length; i++)
73              {
74                  this[i] = input[i];
```

```
            }
        }

        public void Print()
        {
            if (arr.Length == 0)
            {
                Console.WriteLine("(массив пуст)");
                return;
            }
            for (int i = 0; i < Len; i++)
            {
                Console.WriteLine($"{this[i]} ");
            }
        }

        public override string ToString()
        {
            string res = "";
            for (int i = 0; i < Len; i++)
            {
                if (i > 0)
                {
                    res += ", ";
                }
                res += this[i].ToString();
            }
            return res.ToString();
        }
        public Time min()
        {
            if (Count == 0)
            {
                throw new InvalidOperationException("Коллекция пуста");
            }
            Time min = this[0];
            for (var i = 0; i < Len; ++i)
            {
                if (this[i] < min)
                {
                    min = this[i];
                }
            }
            return min;
        }
    }
}
```

3.3 Файл UserInterface.cs

```
using System;

namespace lab_10
{
    internal static class UserInterface
    {

        public static bool selectMenu(out int inputInt)
        {
            Console.Clear();
            Console.WriteLine("1. Задача 1 (класс Time, вычитание минут)");
            Console.WriteLine("2. Задача 2 (перегрузка операций)");
            Console.WriteLine("3. Задача 3 (коллекция TimeArray, поиск минимума)");
            Console.Write("Выберите пункт 1-3 или введите любое другое значение для выхода: ");
            string? input = Console.ReadLine();
            return int.TryParse(input, out inputInt);
        }

        public static int selectInitMethod()
        {
            Console.Clear();
            Console.WriteLine("Варианты заполнения массива:");
            Console.WriteLine("1. Массив из 8 значений по умолчанию");
            Console.WriteLine("2. Массив из случайных значений");
            Console.WriteLine("3. Заполнение вручную");
            Console.Write("Введите значение от 1 до 3: ");
            return readInt((res) => (res >= 1 && res <= 3));
        }


        public static int readInt(Func<int, bool> condition)
        {
            while (true)
            {
                string? input = Console.ReadLine();
                if (int.TryParse(input, out int res) && condition(res))
                {
                    return res;
                }
                Console.Write("Ошибка! Введите заново: ");
            }
        }

        public static void readHoursAndMinutes(out int hours, out int minutes)
```

```
        {
            Console.WriteLine("Введите часы и минуты через пробел: ");
            while (true)
            {
                string input = Console.ReadLine() ?? string.Empty;
                string[] parts = input.Split((char[]?)null,
    StringSplitOptions.RemoveEmptyEntries);
                if (parts.Length >= 2
                    && int.TryParse(parts[0], out hours)
                    && int.TryParse(parts[1], out minutes)
                    && hours >= 0
                    && minutes >= 0
                    && minutes <= 59)
                {
                    return;
                }
                Console.Write("Ошибка! Введите часы и минуты повторно
    через пробел: ");
            }
        }

        public static Time initTimeFromKeyboard()
        {
            readHoursAndMinutes(out int h, out int m);
            return new Time(h, m);
        }

        public static void fillArrFromKeyboard(Time[] arr)
        {
            for (int i = 0; i < arr.Length; i++)
            {
                Console.WriteLine($"Элемент с индексом {i}:");
                readHoursAndMinutes(out int h, out int m);
                arr[i] = new Time(h, m);
            }
        }
    }
}
```

3.4 Файл Program.cs

```
using lab_10;

int select;
bool parsed;

Time first;
Time second;
Time copy;
```

```
9
10 do
11 {
12     parsed = UserInterface.selectMenu(out select);
13     if (!parsed)
14     {
15         break;
16     }
17
18     switch (select)
19     {
20         case 1:
21             {
22                 initObjects();
23                 task1();
24                 break;
25             }
26         case 2:
27             {
28                 initObjects();
29                 task2();
30                 break;
31             }
32         case 3:
33             task3();
34             break;
35     }
36 } while (select >= 1 && select <= 3);
37
38 void initObjects()
39 {
40     Console.WriteLine("Инициализация объектов времени:");
41     first = new Time();
42     Console.WriteLine($"Первое время (конструктор без параметров): {first
       }");
43
44     second = UserInterface.initTimeFromKeyboard();
45     Console.WriteLine($"Второе время (введено с клавиатуры): {second}");
46
47     copy = new Time(second);
48     Console.WriteLine($"Копия второго времени (конструктор копирования):
       {copy}");
49     Console.WriteLine();
50 }
51
52
53 void task1()
54 {
55     first.Minutes = 1;
56
```

```
57      Console.WriteLine("Значения объектов класса:");
58      first.Print("Первое время:");
59      second.Print("Второе время:");
60      copy.Print("Копия второго времени:");
61
62      Console.Write("Введите количество минут для вычитания: ");
63      int minus = UserInterface.readInt((res) => (res >= 0));
64
65      try
66      {
67          Time afterMethod = copy.SubtractMinutes(minus);
68          Console.WriteLine($"Метод класса: {copy} - {minus} = {afterMethod
        }");
69
70          Time afterStatic = Time.Subtract(afterMethod, minus);
71          Console.WriteLine($"Статическая функция: {afterMethod} - {minus}
        = {afterStatic}");
72      }
73      catch (InvalidOperationException ex)
74      {
75          Console.WriteLine($"Ошибка: {ex.Message}");
76      }
77
78      Console.WriteLine($"Количество созданных объектов Time: {Time.Count}"
        );
79      pause();
80  }
81
82  void task2()
83  {
84      Console.WriteLine("Унарные операции:");
85      string before = copy.ToString();
86      try
87      {
88          copy--;
89          Console.WriteLine($"Вычитание минуты (--): {before} -> {copy}");
90      }
91      catch (InvalidOperationException ex)
92      {
93          Console.WriteLine($"Ошибка: {ex.Message}");
94      }
95
96      Time zeroed = -second;
97      Console.WriteLine($"Сброс в ноль (унарный -): {second} -> {zeroed}");
98
99      Console.WriteLine();
100     Console.WriteLine("Преобразования типов:");
101     printConversion("Первое время", first);
102     printConversion("Второе время", second);
103     printConversion("Копия второго времени", copy);
```

```
104
105     Console.WriteLine();
106     Console.WriteLine("Сравнение объектов:");
107     printComparison("1 и 2", first, second);
108     printComparison("1 и копия 2", first, copy);
109     printComparison("2 и копия 2", second, copy);
110     printComparison("2 и новая копия 2", second, new Time(second));
111
112     pause();
113 }
114
115 void task3()
116 {
117     int fillMethod = UserInterface.selectInitMethod();
118     TimeArray array;
119     switch (fillMethod)
120     {
121         case 1:
122             array = new TimeArray();
123             break;
124         case 2:
125             Console.Write("Введите длину массива: ");
126             array = new TimeArray(UserInterface.readInt(res => res > 0));
127             break;
128         default:
129             Console.Write("Введите длину массива: ");
130             int length = UserInterface.readInt(res => res > 0);
131             Time[] input = new Time[length];
132             UserInterface.fillArrFromKeyboard(input);
133             array = new TimeArray(input);
134             break;
135     }
136
137     Console.WriteLine();
138     Console.WriteLine("Полученный массив:");
139     array.Print();
140
141     try
142     {
143         Console.WriteLine();
144         Console.WriteLine($"Минимальный элемент: {min(array)}");
145     } catch (InvalidOperationException ex)
146     {
147         Console.WriteLine(ex.Message);
148     }
149
150     Console.WriteLine("Изменение элемента через индексатор: array[0] =
    01:15");
151     array[0] = new Time(1, 15);
152     array.Print();
```

```
153
154     Console.WriteLine("Проверка выхода индекса за границы массива:");
155     try
156     {
157         Console.WriteLine($"array[{array.Len}] = {array[array.Len]}");
158     }
159     catch (ArgumentOutOfRangeException ex)
160     {
161         Console.WriteLine($"Перехвачено ожидаемое исключение: {ex.Message
    }");
162     }
163
164     Console.WriteLine($"Создано объектов Time: {Time.Count}, объектов
    TimeArray: {TimeArray.Count}");
165     pause();
166 }
167
168
169 void printConversion(string label, Time time)
170 {
171     Console.WriteLine($"{label} {time}: int -> {(int)time} (часы), bool
    -> {(bool)time} (признак ненулевого времени)");
172 }
173
174 void printComparison(string label, Time left, Time right)
175 {
176     Console.WriteLine($"{label}: == {left == right} ({(left == right ? "
    равны" : "не равны")}), " +
177                       $"!= {left != right} ({(left != right ? "не равны"
    : "равны")})");
178 }
179
180 void pause()
181 {
182     Console.Write("Введите любое значение для выхода: ");
183     Console.ReadLine();
184 }
185
186 public Time min(TimeArray arr)
187 {
188     if (arr.Len == 0)
189     {
190         throw new InvalidOperationException("Коллекция пуста");
191     }
192     Time min = arr[0];
193     for (var i = 0; i < arr.Len; ++i)
194     {
195         if (arr[i] < min)
196         {
197             min = arr[i];
```

```
198             }
199         }
200         return min;
201 }
```